

QUEUE

Q9)

Question:

DS	Session	Queue	Question Information	Level 1	Challenge 59
Question description On the last day of the semester, Shahid's students were talking and playing very loudly to the delight of the end of the semester. The principal of the college severely reprimanded Shahid. But he decided to engage them in a different activity without getting angry at the students. So Shahid gave his students to solve the task such that, Circular Queue using Linked List and dequeue two elements from circular queue. there is no memory waste while using Circular Queue, it is preferable than using a regular queue. Because linked lists allow for dynamic memory allocation, they are simple to build. Circular Queue implementation using linked list is identical to circular linked list except that circular Queue has two pointers front and back whereas circular linked list only has one pointer head. Final Year students were lacking the idea to solve the problem. Being an exciting youngster can you solve it? Function Description					
enqueue(34) 34 ptr Front/Rear enqueue(33) 34 ptr Front 33 ptr Rear enqueue(22) 34 ptr Front 33 ptr 22 ptr Rear enqueue(76) 34 ptr Front 33 ptr 22 ptr 76 ptr Rear enqueue(32) 34 ptr Front 33 ptr 22 ptr 76 ptr 32 ptr Removing the elements from Queue dequeue() 33 ptr Front 22 ptr 76 ptr 32 ptr Rear Constraints: 0 < size < 100 0 < data < 1000					

PROGRAM:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct Node {
5      int data;
6      struct Node *next;
7  };
8
9  struct Node *front = NULL;
10 struct Node *rear = NULL;
11
12 void enqueue(int data) {
13     struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
14     newNode->data = data;
15
16     if (front == NULL) {
17         front = rear = newNode;
18         rear->next = front;
19     } else {
20         newNode->next = front;
21         rear->next = newNode;
22         rear = newNode;
23     }
24 }
25
26 void dequeue() {
27     struct Node *temp;
28
29     if (front == NULL)
30         return;
31
32     if (front == rear) {
33         free(front);
34         front = rear = NULL;
35     } else {
36         temp = front;
37         front = front->next;
38         rear->next = front;
39         free(temp);
40     }
41 }
42
43 void display() {
44     struct Node *temp;
45
46     if (front == NULL)
47         return;
48
49     temp = front;
50
51     do {
52         printf("%d ", temp->data);
53         temp = temp->next;
54     } while (temp != front);
55
56     printf("\n");
57 }
58
59 int main() {
60     int n, data;
61
62     scanf("%d", &n);
63
64     for (int i = 0; i < n; i++) {
65         scanf("%d", &data);
66         enqueue(data);
67     }
68
69     display();
70
71     dequeue();
72     display();
73
74     dequeue();
75     display();
76
77     return 0;
78 }
```

OUTPUT:

TEST CASE 1:

```
5
9 1 5 3 5
9 1 5 3 5
1 5 3 5
5 3 5
```

TEST CASE 2:

```
8
1 8 6 4 9 5 3 5
1 8 6 4 9 5 3 5
8 6 4 9 5 3 5
6 4 9 5 3 5
```